Homework 1

Computation of Lyapunov orbits in the PCR3BP: L_1 and L_2 families of the Earth–Moon system and pairs with equal Jacobi constant

Francesco Caroletta
Interplanetary Trajectories / 2064179

November 3, 2026

Abstract

The Lyapunov families around the collinear points L_1 and L_2 of the Earth–Moon system are computed in the planar circular restricted three-body problem (PCR3BP). Small-amplitude orbits are built from the linearized solution and corrected with a single-shooting scheme that exploits the symmetry of the problem. The families are then extended with a natural continuation on x_0 with adaptive step, yielding 25 orbits for L_1 and 40 for L_2. Finally, a bisection on the natural parameter is used to find three L_1/L_2 orbit pairs with the same Jacobi constant in the range $C \in [3.1370, 3.1493]$, with $|\Delta C| \leq 1.1 \cdot 10^{-11}$ for every pair and closure errors below $5 \cdot 10^{-11}$.

Contents

1 Introduction **3**

2 Dynamical model **3**
2.1 Equations of motion of the PCR3BP 3
2.2 Jacobi constant 3
2.3 Collinear points 3
2.4 Symmetry 4

3 State transition matrix **4**

4 Linearized motion near the collinear points **4**

5 Nonlinear Lyapunov orbits: TPBVP formulation **5**
5.1 Periodicity condition 5
5.2 Problem definition 5
5.3 Newton correction 5
5.4 Explicit computation of DF 6

6 Continuation schemes **6**
6.1 Natural continuation (NC) 6
6.2 Pseudo-arclength continuation (PAC) 6

7 Numerical implementation **7**
7.1 Parameters . . . 7
7.2 Integration . . . 7
7.3 Initial orbit . . . 7
7.4 Natural continuation with adaptive step . . . 7
7.5 Equal-Jacobi pairs: bisection . . . 8
7.6 Verification . . . 8

8 Results **9**
8.1 Libration points and linear analysis . . . 9
8.2 Small-amplitude orbits . . . 9
8.3 Lyapunov families . . . 9
8.4 L_1/L_2 pairs with equal Jacobi constant . . . 11
8.5 Verification of periodicity . . . 12

9 Discussion and conclusions **12**

A MATLAB code **13**

1 Introduction

Lyapunov orbits are planar, symmetric, periodic solutions of the CR3BP located around the three collinear points. They are the smallest planar periodic orbits near the libration points and, by increasing their amplitude, they form a continuous family for each point. They can be computed easily and, thanks to their instability, they give useful insight into the natural transport of matter inside binary planetary systems that can be approximated with the CR3BP.

This work focuses on the planar version (PCR3BP) and on the Earth–Moon system. In the neighborhood of the collinear points the planar phase space is of the type *saddle* × *center*: one positive and one negative real eigenvalue and a pair of purely imaginary ones (*saddle* × *center* × *center* in 3D).

The goals of the homework are:

1. to compute the Lyapunov families of L_1 and L_2 starting from a small-amplitude orbit, using a numerical continuation scheme (NC or PAC);
2. to compute, by bisection, L_1/L_2 orbit pairs with the *same* Jacobi constant in the range common to the two families, with at least two or three pairs in $C \in [3.1370, 3.1493]$.

2 Dynamical model

2.1 Equations of motion of the PCR3BP

In the non-dimensional synodic frame the two primaries, Earth and Moon, are fixed on the x-axis at $-\mu$ and $1-\mu$, with

$$\mu = \frac{\mu_{\text{Moon}}}{\mu_{\text{Earth}} + \mu_{\text{Moon}}}. \tag{1}$$

The equations of motion of the massless body are

$$\begin{bmatrix} \ddot{x} \\ \ddot{y} \end{bmatrix} = 2 \begin{bmatrix} \dot{y} \\ -\dot{x} \end{bmatrix} + \begin{bmatrix} U_x \\ U_y \end{bmatrix}, \qquad U = \frac{x^2+y^2}{2} + \frac{1-\mu}{r_1} + \frac{\mu}{r_2}, \tag{2}$$

where $r_1 = \sqrt{(x+\mu)^2+y^2}$ and $r_2 = \sqrt{(x+\mu-1)^2+y^2}$ are the distances from Earth and Moon. With $u = \dot{x}$ and $v = \dot{y}$ the first-order form is

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} u \\ v \\ 2v + U_x \\ -2u + U_y \end{bmatrix}, \qquad \boldsymbol{x} = [x,\, y,\, u,\, v]^T. \tag{3}$$

2.2 Jacobi constant

The system admits a first integral, the Jacobi constant, defined here as

$$C = 2U - (u^2 + v^2). \tag{4}$$

Lyapunov orbits are initialized on the x-axis with $y_0 = 0$, $u_0 = 0$, hence $C = 2U(x_0, 0) - v_0^2$. The zero-velocity curves (ZVC), $2U(x, y) = C$, bound the region where motion is admissible for a given value of C.

2.3 Collinear points

L_1, L_2 and L_3 lie on the x-axis and are the zeros of $U_x(x, 0) = 0$. In the code L_1 and L_2 are found with `fzero` in the intervals [0.5, 0.95] (between Earth and Moon) and [1.0, 1.4] (beyond the Moon), respectively.

2.4 Symmetry

Equations (2) are invariant under the transformation $(y, t) \to (-y, -t)$. Consequently, if $\phi(\boldsymbol{x}_0^+, t)$ is the trajectory starting from $\boldsymbol{x}_0^+ = [x_0,\, y_0,\, u_0,\, v_0]^T$, the trajectory $\phi(\boldsymbol{x}_0^-, -t)$ with $\boldsymbol{x}_0^- = [x_0,\, -y_0,\, -u_0,\, v_0]^T$ is its mirror image with respect to the x-axis. Trajectories are therefore symmetric with respect to time and to the xz-plane.

3 State transition matrix

For an autonomous system $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$ with $\boldsymbol{x}(0) = \boldsymbol{x}_0$ the flow is

$$\boldsymbol{x}(t) = \phi(\boldsymbol{x}_0, t) = \boldsymbol{x}_0 + \int_0^t \boldsymbol{f}(\boldsymbol{x}(\tau))\, \mathrm{d}\tau. \tag{5}$$

A displaced trajectory $\boldsymbol{x}^*(t) = \boldsymbol{x}(t) + \delta\boldsymbol{x}(t)$ satisfies the equations of motion, and to first order

$$\delta\dot{\boldsymbol{x}}(t) = \frac{\partial \boldsymbol{f}(\boldsymbol{x}(t))}{\partial \boldsymbol{x}}\, \delta\boldsymbol{x}(t) = A(\boldsymbol{x}(t))\, \delta\boldsymbol{x}(t). \tag{6}$$

Expanding the flow with respect to the initial condition gives

$$\delta\boldsymbol{x}(t) = \frac{\partial \phi(\boldsymbol{x}_0, t)}{\partial \boldsymbol{x}_0}\, \delta\boldsymbol{x}_0 = \Phi(t, 0)\, \delta\boldsymbol{x}_0, \tag{7}$$

where $\Phi(t, 0)$ is the *state transition matrix* (STM). Differentiating in time and comparing with Eq. (6) yields the variational equation

$$\dot{\Phi} = A(\boldsymbol{x}(t))\, \Phi(t, 0), \qquad \Phi(0, 0) = \mathbb{I}. \tag{8}$$

In practice the STM is integrated together with the state, in an augmented vector of dimension $4 + 16 = 20$.

4 Linearized motion near the collinear points

Let $\boldsymbol{x}_L$ be the state of a collinear point and $\boldsymbol{x} = \boldsymbol{x}_L + \delta\boldsymbol{x}$. The linearized equations are $\delta\dot{\boldsymbol{x}} = A(\boldsymbol{x}_L)\, \delta\boldsymbol{x}$, with

$$A(\boldsymbol{x}_L) = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ U_{xx} & U_{xy} & 0 & 2 \\ U_{xy} & U_{yy} & -2 & 0 \end{bmatrix}_{\boldsymbol{x}_L} \tag{9}$$

a constant matrix. For the three collinear points its eigenvalues are

$$\lambda_u > 0, \qquad \lambda_s = -\lambda_u < 0, \qquad \lambda_{c1,c2} = \pm j\omega. \tag{10}$$

The associated eigenvectors $\boldsymbol{E}_u$, $\boldsymbol{E}_s$, $\boldsymbol{E}_{c1}, \boldsymbol{E}_{c2}$ span, respectively, the divergent motion, the convergent motion and the periodic motion, the latter being possible only in the center subspace.

With the basis matrix $Z = [\boldsymbol{E}_u\ \boldsymbol{E}_s\ \mathrm{Re}(\boldsymbol{E}_{c1})\ \mathrm{Im}(\boldsymbol{E}_{c1})]$ and the coordinates $\boldsymbol{e} = [\xi\ \eta\ \alpha\ \beta]^T$, such that $\delta\boldsymbol{x} = Z\boldsymbol{e}$, the system becomes $\dot{\boldsymbol{e}} = Z^{-1}AZ\,\boldsymbol{e} = D\boldsymbol{e}$, with real Jordan form

$$D = \begin{bmatrix} \lambda_u & 0 & 0 & 0 \\ 0 & \lambda_s & 0 & 0 \\ 0 & 0 & 0 & \omega \\ 0 & 0 & -\omega & 0 \end{bmatrix}. \tag{11}$$

The solution for $\boldsymbol{e}_0 = [\xi_0\ \eta_0\ \alpha_0\ \beta_0]^T$ is

$$\begin{cases} \xi(t) = \xi_0 e^{\lambda_u t} \\ \eta(t) = \eta_0 e^{\lambda_s t} \\ \alpha(t) = \alpha_0 \cos\omega t + \beta_0 \sin\omega t \\ \beta(t) = \beta_0 \cos\omega t - \alpha_0 \sin\omega t \end{cases} \tag{12}$$

The periodic motion always has period $T = 2\pi/\omega$, regardless of the initial condition. Setting $\boldsymbol{e}_0 = [0\ 0\ \alpha_0\ 0]^T$ removes the unstable and stable components and gives the linearized Lyapunov orbit:

$$\boldsymbol{x}(t) = \boldsymbol{x}_L + \alpha_0 \Big(\cos(\omega t)\,\mathrm{Re}(\boldsymbol{E}_{c1}) - \sin(\omega t)\,\mathrm{Im}(\boldsymbol{E}_{c1}) \Big). \tag{13}$$

This approximation only describes small-amplitude orbits: propagated in the nonlinear model it eventually diverges, so it must be corrected.

5 Nonlinear Lyapunov orbits: TPBVP formulation

5.1 Periodicity condition

By the symmetry of Section 2.4, a sufficient condition for periodicity in the PCR3BP is to have *two perpendicular crossings of the x-axis*. Lyapunov orbits satisfy it, since the first two components of $\mathrm{Re}(\boldsymbol{E}_{c1})$ and $\mathrm{Im}(\boldsymbol{E}_{c1})$ are parallel to the x and y axes. A single-shooting scheme can therefore be used: the orbit is initialized at one crossing and the other is targeted by adjusting the initial conditions.

5.2 Problem definition

The orbit starts perpendicularly on the x-axis, $\boldsymbol{x}_0 = [x_0,\, 0,\, 0,\, v_0]^T$, so the unknowns are x_0, v_0 and the period T:

$$X = [x_0,\ v_0,\ T]^T. \tag{14}$$

At $t = T/2$ the orbit must cross the x-axis perpendicularly again, so the constraint vector is

$$F(X) = \begin{bmatrix} y(T/2) \\ u(T/2) \end{bmatrix} = \mathbf{0}. \tag{15}$$

Since T is unknown, time is rescaled as $t = T\tau$, so that every orbit is integrated on the fixed interval $\tau \in [0, 1]$. The scaled dynamics and variational equations become

$$\boldsymbol{x}' = T\,\boldsymbol{f}(\boldsymbol{x}(\tau)), \tag{16}$$

$$\Phi' = T\,A(\boldsymbol{x}(\tau))\,\Phi, \tag{17}$$

where the prime denotes the derivative with respect to τ. The constraint is evaluated at $\tau = 1/2$.

5.3 Newton correction

If $X^* = X + \delta X$ is the sought solution, linearizing the constraint vector gives

$$0 = F(X^*) \simeq F(X) + DF(X)\,\delta X. \tag{18}$$

This system is underdetermined (2 equations, 3 unknowns); using the pseudo-inverse,

$$\delta X = -(DF^T DF)^{-1} DF^T F(X). \tag{19}$$

The process is repeated until a stopping criterion is met, e.g. a tolerance on $\|F\|$.

5.4 Explicit computation of DF

The periodic orbit is the flow $\phi(\tau, \boldsymbol{x}_0, T)$. Its partial derivatives are

- with respect to the initial condition: the STM $\Phi(\tau)$;
- with respect to the period: $\boldsymbol{w}(\tau) = \partial\phi/\partial T$.

Differentiating $\phi' = T\boldsymbol{f}(\phi)$ with respect to T gives

$$\boldsymbol{w}'(\tau) = \boldsymbol{f} + T\,A(\tau)\,\boldsymbol{w}(\tau). \tag{20}$$

The homogeneous part vanishes because $\boldsymbol{w}(0) = \boldsymbol{0}$ (at $\tau = 0$ a variation of T does not affect the orbit), and the particular integral, obtained with the variation-of-constants formula, gives

$$\boldsymbol{w}(\tau) = \Phi(\tau) \int_0^\tau \Phi(s)^{-1} \boldsymbol{f}(\phi(s, \boldsymbol{x}_0, T))\,\mathrm{d}s = \tau\,\Phi(\tau)\,\boldsymbol{f}(\boldsymbol{x}_0). \tag{21}$$

The Jacobian of F is therefore made of

- $\partial F/\partial[x_0, v_0]$: the 2×2 matrix given by rows 2 and 3 and columns 1 and 4 of $\Phi(1/2)$;
- $\partial F/\partial T$: the 2×1 vector given by components 2 and 3 of $\boldsymbol{w}(1/2)$.

Explicitly,

$$DF = \begin{bmatrix} \phi_{yx} & \phi_{yv} & \frac{1}{2}\,\Phi(2,:)\,\boldsymbol{f}(\boldsymbol{x}_0) \\ \phi_{ux} & \phi_{uv} & \frac{1}{2}\,\Phi(3,:)\,\boldsymbol{f}(\boldsymbol{x}_0) \end{bmatrix}_{\tau=1/2}. \tag{22}$$

6 Continuation schemes

The solutions of the TPBVP belong to a smooth branch that can be parameterized: once a point of the branch has been found, a nearby one can be computed with a numerical continuation scheme.

6.1 Natural continuation (NC)

NC varies in a controlled way a single component of the design vector, the *natural parameter*, assuming that the others are functions of it. Here x_0 is chosen, so $v_0 = v_0(x_0)$ and $T = T(x_0)$. Given the orbit X_1, the next one has $x_{0,2} = x_{0,1} + \delta x_0$, where x_0 is now a known fixed quantity and is removed from the unknowns together with the first column of DF. The guesses $v_{0,2} = v_{0,1}$ and $T_2 = T_1$ provide the initial condition for Newton's method, which converges if δx_0 is small enough to remain in the basin of attraction. NC works when the design vector is a monotonic function of the natural parameter, as is the case for Lyapunov orbits. A step that is too large may make the shooting converge to a different family of planar symmetric periodic orbits.

6.2 Pseudo-arclength continuation (PAC)

PAC parameterizes the branch $X(s)$ with an arclength parameter s and uses the tangent vector $\boldsymbol{\nu}$, which belongs to the nullspace of $DF(X_1)$. The new orbit is sought at distance Δs from X_1, with guess $X_2 = X_1 + \Delta s\,\boldsymbol{\nu}_1$, by solving

$$G(X) = \begin{bmatrix} F(X) \\ \boldsymbol{\nu}_1^T (X - X_1) - \Delta s \end{bmatrix} = \boldsymbol{0}, \qquad DG = \begin{bmatrix} DF \\ \boldsymbol{\nu}_1^T \end{bmatrix}. \tag{23}$$

The sign of the tangent vector must be checked so that $\boldsymbol{\nu}_k$ and $\boldsymbol{\nu}_{k-1}$ are not opposite, otherwise the scheme converges back to a previous solution.

Adopted scheme. In this work **natural continuation on** x_0 was used, with an adaptive step (Section 7). For Lyapunov orbits v_0 and T are monotonic functions of x_0 along the family, so NC is appropriate and does not require the nullspace of DF.

7 Numerical implementation

The code is written in MATLAB; the full listing is given in Appendix A.

7.1 Parameters

Parameter	Value
μ_{Earth}	$3.98600436 \cdot 10^5$ km^3/s^2
μ_{Moon}	$4.902799 \cdot 10^3$ km^3/s^2
Earth–Moon distance	384400 km
$\mu = \mu_{\text{Moon}}/(\mu_{\text{Earth}} + \mu_{\text{Moon}})$	$1.215058164279650 \cdot 10^{-2}$
`ode113` RelTol / AbsTol	$3 \cdot 10^{-14}$ / $1 \cdot 10^{-14}$

Table 1: Model and integrator parameters.

7.2 Integration

State and STM are integrated together (20 equations) with `ode113` on the scaled domain $\tau \in [0, 0.5]$ for the corrections and $\tau \in [0, 1]$ for the verification, using Eqs. (16)–(17). The Jacobian A is computed analytically from the second derivatives of the potential U_{xx}, U_{yy}, U_{xy}. To express periods in days, the time unit $TU = \sqrt{R^3/(\mu_{\text{Earth}} + \mu_{\text{Moon}})} \simeq 4.34$ days is used.

7.3 Initial orbit

For each point, $\boldsymbol{x}_L = [x_L, 0, 0, 0]^T$ is found with `fzero`. From the eigenvalues of $A(\boldsymbol{x}_L)$ the frequency ω and the eigenvector $\boldsymbol{E}_{c1}$ are extracted from the imaginary pair. The initial guess comes from Eq. (13) at $t = 0$ with $\alpha_0 = 10^{-4}$:

$$\boldsymbol{x}_0 = \boldsymbol{x}_L + \alpha_0 \,\mathrm{Re}(\boldsymbol{E}_{c1}), \qquad X^{(0)} = [x_0,\ v_0,\ 2\pi/\omega]^T, \tag{24}$$

keeping only x_0 and v_0 of $\boldsymbol{x}_0$ and imposing $y_0 = u_0 = 0$. The orbit is then corrected with the three-unknown Newton scheme of Eq. (18), solving the underdetermined system with the MATLAB operator \, and stopping when $|F_i| < 10^{-12}$ for $i = 1, 2$.

7.4 Natural continuation with adaptive step

The natural parameter is x_0, incremented from the initial orbit moving away from the libration point (direction $s = \mathrm{sign}(x_0 - x_L)$). At step k one sets $x_{0,k+1} = x_{0,k} + s\,\delta x$ and solves the shooting problem with fixed x_0 and unknowns $[v_0, T]$: removing the first column of DF leaves a 2×2 system,

$$\begin{bmatrix} \delta v_0 \\ \delta T \end{bmatrix} = -DF^{-1}_{(:,2:3)}\, F. \tag{25}$$

Details of the scheme:

- **Initial guess.** From the second orbit on, a secant predictor is used, $p^{(0)}_{k+1} = p_k + (p_k - p_{k-1})\,\delta x/h_{k-1}$, with $p = [v_0, T]$ and $h_{k-1} = |x_{0,k} - x_{0,k-1}|$. For the first step, v_0 is scaled linearly with $(x_0 - x_L)$ (linear regime) and T is kept.
- **Shooting.** Tolerance $\|F\|_\infty < 10^{-12}$, at most 20 iterations, with rejection of non-physical periods ($T \le 0$).
- **Consistency check.** The orbit is discarded if $|T_{k+1} - T_k| > 0.1\,T_k$, to avoid jumps to other families or to the trivial solution.

- **Adaptive step.** On failure the step is halved and the attempt is repeated from the same orbit; on success the step is multiplied by 1.5 up to a maximum $\delta x_{\max}$. If $\delta x < \delta x_{\min}$ the continuation stops.
- **Stopping.** The continuation ends when $C < C_{\min}$ or when the maximum number of orbits is reached.

Parameter	Symbol	Value
Initial step	δx_0	10^{-5}
Maximum step	$\delta x_{\max}$	$2 \cdot 10^{-3}$
Minimum step	$\delta x_{\min}$	10^{-9}
Maximum number of orbits	$N_{\max}$	3000
Minimum Jacobi constant	$C_{\min}$	3.130
Shooting tolerance	$\|F\|_\infty$	10^{-12}

Table 2: Natural continuation and shooting parameters.

The maximum step $\delta x_{\max}$ keeps adjacent orbits close enough for the subsequent bisection to operate on intervals where the solution varies smoothly.

7.5 Equal-Jacobi pairs: bisection

Given a target value C^* in the range common to both families, for each family the pair of consecutive orbits (a, b) such that $(C_a - C^*)(C_b - C^*) \leq 0$ is located. The bisection is then applied to the natural parameter x_0:

1. midpoint $x_m = (x_{0,a} + x_{0,b})/2$;
2. guess for $[v_0, T]$ as the average of those of the two endpoints;
3. recomputation of the orbit at x_m with the fixed-x_0 shooting and evaluation of $C_m = 2U(x_m, 0) - v_m^2$;
4. update of the endpoint so that the bracket of C^* is preserved.

The stopping criterion is $|C_m - C^*| < 10^{-11}$ (or an interval width in x_0 below 10^{-15}), with at most 100 iterations. Since the criterion is applied to each family separately, the mismatch within a pair is guaranteed only to be below $2 \cdot 10^{-11}$. The targets are

$$C^* \in \{3.1380,\ 3.1420,\ 3.1470\}, \tag{26}$$

all inside the required range [3.1370, 3.1493].

7.6 Verification

For each orbit found, a full period is integrated and the following are computed:

- the closure error $\|\boldsymbol{x}(\tau = 1) - \boldsymbol{x}(\tau = 0)\|$;
- the maximum drift of the Jacobi constant, $\max |C(\tau) - C(0)|$.

8 Results

8.1 Libration points and linear analysis

Table 3 reports the libration points and the eigenvalues of $A(\boldsymbol{x}_L)$. In dimensional terms, L_1 lies about 58 019 km from the Moon (towards the Earth) and L_2 about 64 515 km from the Moon (beyond it).

	L_1	L_2
x_L	0.836915145291	1.155682150188
$C_L = 2U(x_L, 0)$	3.18834108	3.17216043
$\lambda_u = -\lambda_s$	2.9320558845	2.1586743565
ω	2.3343858541	1.8626458833
$T_{\text{lin}} = 2\pi/\omega$	2.6915795844	3.3732580967

Table 3: Collinear points and eigenvalues of $A(\boldsymbol{x}_L)$ ($\lambda_{c1,c2} = \pm j\omega$).

8.2 Small-amplitude orbits

Starting from the linear guess with $\alpha_0 = 10^{-4}$, the Newton scheme converged after one correction for both points (the second evaluation of the constraint already satisfied $|F_i| < 10^{-12}$). The resulting orbits are reported in Table 4; the converged periods differ from $2\pi/\omega$ by about $2 \cdot 10^{-8}$ for L_1, as expected for such a small amplitude.

	L_1	L_2
x_0	0.836904570310	1.155666789230
$x_0 - x_L$	$-1.0575 \cdot 10^{-5}$	$-1.5361 \cdot 10^{-5}$
v_0	$8.854354752579 \cdot 10^{-5}$	$8.333085513297 \cdot 10^{-5}$
T	2.691579607523	3.373258106559
C	3.1883410746	3.1721604245
Newton corrections	1	1

Table 4: Small-amplitude Lyapunov orbits after the correction.

8.3 Lyapunov families

The natural continuation produced 25 orbits for L_1 and 40 for L_2 (Table 5). Both families were stopped by the criterion $C < C_{\min} = 3.130$. The Jacobi range common to the two families is [3.1285, 3.1722], which contains the required interval [3.1370, 3.1493].

	L_1	L_2
Number of orbits	25	40
x_0 range	[0.811086, 0.836905]	[1.099848, 1.155667]
C range	[3.127199, 3.188341]	[3.128489, 3.172160]
T range [-]	[2.6916, 2.9581]	[3.3733, 3.4792]
T range [days]	[11.69, 12.85]	[14.65, 15.11]

Table 5: Summary of the computed families.

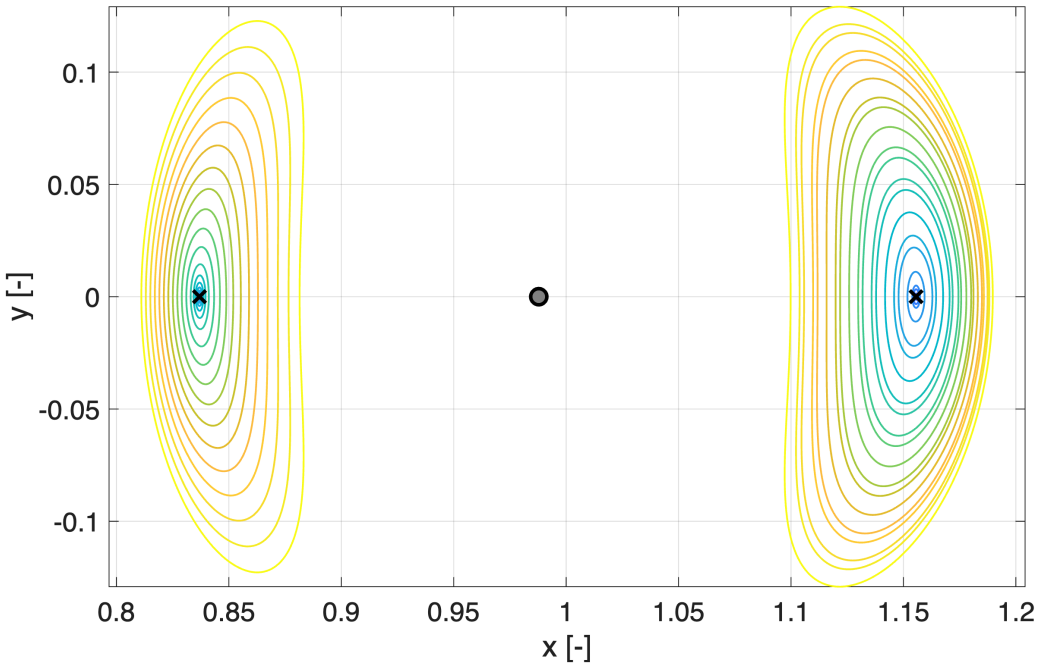


Figure 1: Lyapunov orbits around L_1 and L_2 in the (x, y) synodic plane of the Earth–Moon system. For L_1 all the 25 computed orbits are shown; for L_2, 25 orbits uniformly sampled among the 40 computed. Colors follow the position along the family, from blue (small amplitude, high C) to yellow (large amplitude, low C). The Moon (circle) and the libration points (crosses) are also shown.

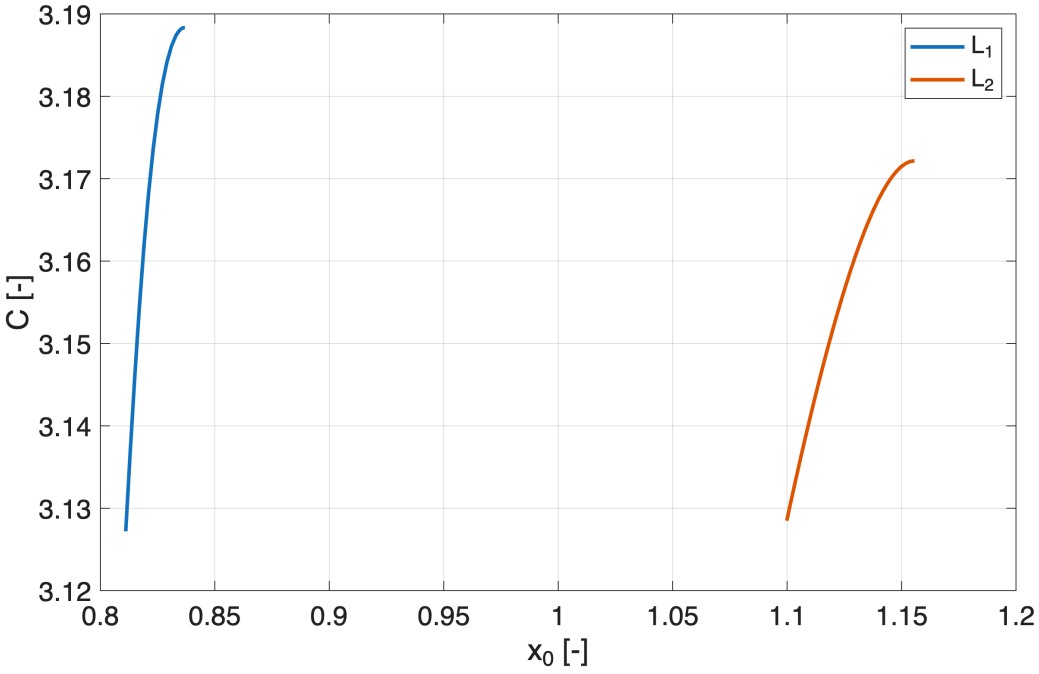


Figure 2: Jacobi constant as a function of x_0 for the two families.

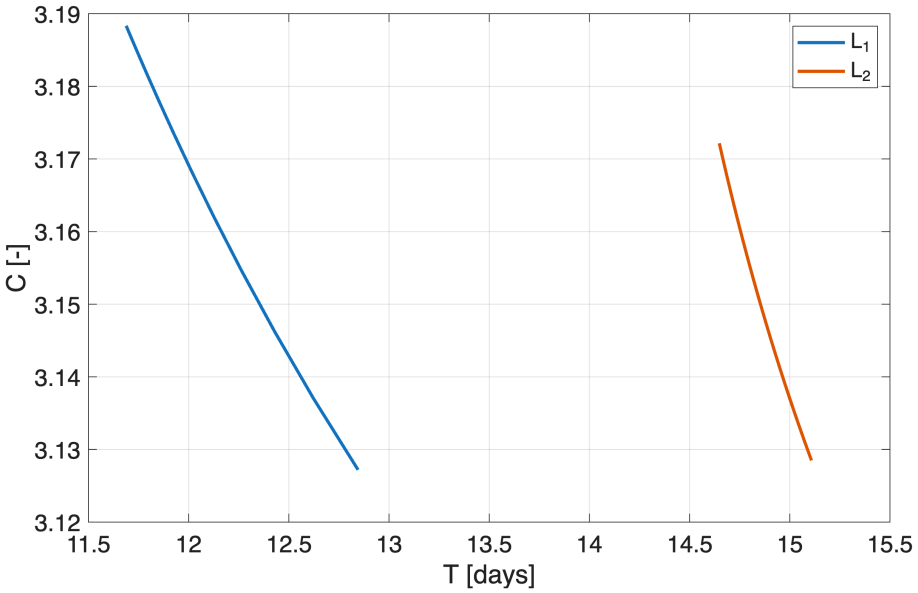


Figure 3: Jacobi constant as a function of the period T (in days) for the two families.

8.4 L_1/L_2 pairs with equal Jacobi constant

The bisection of Section 7.5 gave, for each target C^*, one orbit per family (Tables 6 and 7). In all cases the two Jacobi constants agree to within $|\Delta C| \leq 1.1 \cdot 10^{-11}$ (Table 8).

C^*	x_0	v_0	T	T [d]	C	Bisection it.
3.1380	0.8132887584	0.2369888932	2.90195986	12.6017	3.137999999998	29
3.1420	0.8141442701	0.2275159694	2.88234184	12.5165	3.142000000007	29
3.1470	0.8152591153	0.2150861799	2.85863759	12.4136	3.147000000002	28

Table 6: L_1 orbits at the target Jacobi constants.

C^*	x_0	v_0	T	T [d]	C	Bisection it.
3.1380	1.1074920475	0.2371351631	3.45148938	14.9880	3.138000000003	25
3.1420	1.1109152322	0.2209043364	3.44071194	14.9412	3.141999999996	27
3.1470	1.1154242732	0.1995577370	3.42791109	14.8856	3.147000000006	25

Table 7: L_2 orbits at the target Jacobi constants.

C^*	$\lvert C_{L_1} - C_{L_2} \rvert$
3.1380	$4.88 \cdot 10^{-12}$
3.1420	$1.09 \cdot 10^{-11}$
3.1470	$4.07 \cdot 10^{-12}$

Table 8: Jacobi constant mismatch within each pair.

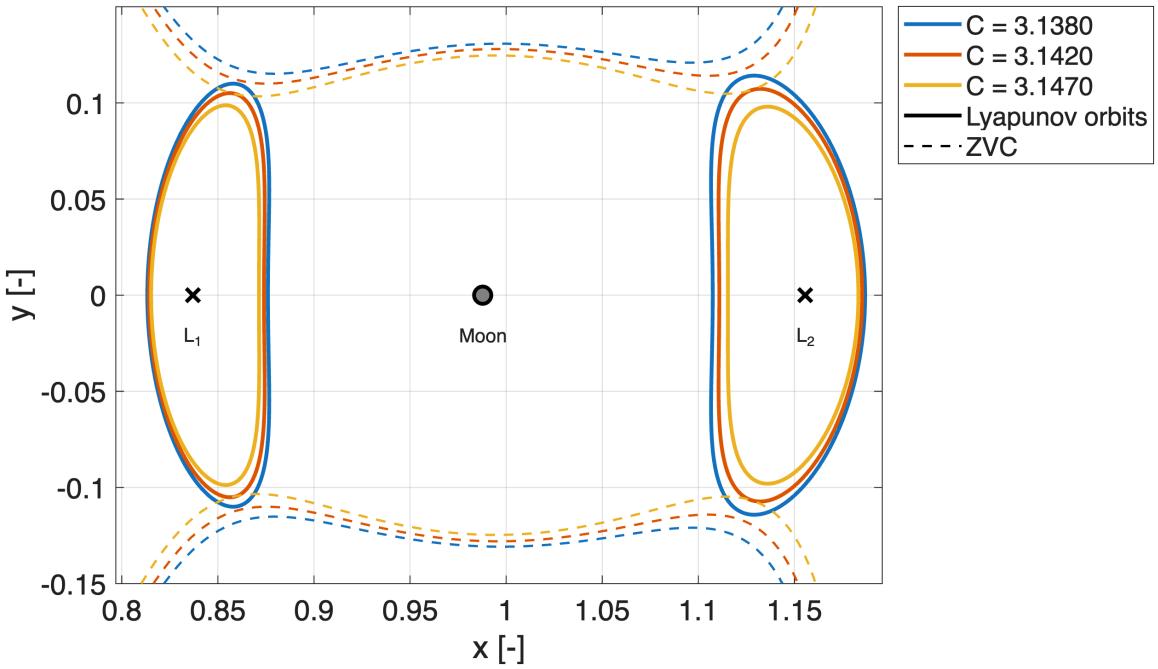


Figure 4: L_1/L_2 orbit pairs with the same Jacobi constant (same color, solid lines) and the corresponding zero-velocity curves (dashed).

8.5 Verification of periodicity

C^*	Point	Closure error	$\max \lvert C - C_0 \rvert$
3.1380	L_1	$9.91 \cdot 10^{-12}$	$1.33 \cdot 10^{-15}$
3.1380	L_2	$2.33 \cdot 10^{-11}$	$2.22 \cdot 10^{-15}$
3.1420	L_1	$1.27 \cdot 10^{-12}$	$1.33 \cdot 10^{-15}$
3.1420	L_2	$4.55 \cdot 10^{-11}$	$1.33 \cdot 10^{-15}$
3.1470	L_1	$1.05 \cdot 10^{-11}$	$1.33 \cdot 10^{-15}$
3.1470	L_2	$1.75 \cdot 10^{-11}$	$2.22 \cdot 10^{-15}$

Table 9: Closure error and conservation of the Jacobi constant over one period.

9 Discussion and conclusions

Families. Along both families the Jacobi constant decreases as the orbit grows, moving x_0 away from the libration point: from $C_{L_1} = 3.1883$ down to 3.1272 for L_1, and from $C_{L_2} = 3.1722$ down to 3.1285 for L_2 (Fig. 2). Both curves flatten as they approach x_L, as expected since in the linear regime $C_L - C$ grows with the square of the amplitude. The L_1 curve is steeper: the mean slope $\Delta C/\Delta x_0$ is about 2.4 for L_1 against 0.78 for L_2, a ratio of about three. The period grows with the amplitude, but by a different amount: from 2.69 to 2.96 (about +10%, i.e. 11.7 to 12.9 days) for L_1, and from 3.37 to 3.48 (about +3%, i.e. 14.7 to 15.1 days) for L_2. The L_2 family also has a longer linear period, consistent with its smaller ω (Table 3), and the two families cover disjoint ranges of period (Fig. 3), in which C decreases almost linearly with T. The continuation never left the monotonic regime in x_0, in agreement with the assumption behind the natural continuation: C is a smooth monotonic function of both x_0 and T along each family, with no visible jump to other families.

Pairs at equal C. All the target values lie below both $C_{L_2} = 3.1722$ and $C_{L_1} = 3.1883$, so the zero-velocity curves are open at both necks (Fig. 4). At the same C the two orbits are quite different. The L_2 orbit has a period about 19–20% longer than the L_1 one (about 14.9 versus 12.5 days), and its initial crossing is displaced from the libration point by about 1.9–2.0 times as much (e.g. $|x_0 - x_L| \simeq 0.0236$ for L_1 and 0.0482 for L_2 at $C^* = 3.1380$). In Figs. 1 and 4 the orbits are elongated along y and, for both points, the side facing the Moon is flatter than the opposite one. The orbits of a pair have a comparable extent in y, while the L_2 orbits are wider in x. For a given C^* the orbits are nested, with larger amplitude for lower C^*, and across the three pairs v_0 and T vary monotonically with C^* in both families.

Accuracy. The bisection stops when each orbit is within 10^{-11} of C^*; in the computed pairs the individual deviations are at most $7 \cdot 10^{-12}$ and the mismatch within a pair is at most $1.1 \cdot 10^{-11}$ (Table 8), reached in 25–29 bisection iterations. The conservation of C over one period is at the level of machine precision ($\lesssim 2.2 \cdot 10^{-15}$), which confirms the quality of the integration with the tolerances of Table 1. The closure error ranges from $1.3 \cdot 10^{-12}$ to $4.6 \cdot 10^{-11}$ (non-dimensional; at most about 2 cm in position). A preliminary run with a shooting tolerance $\|F\|_\infty < 10^{-10}$ gave closure errors of $1.7\text{–}3.3 \cdot 10^{-9}$ on the same pairs; tightening the tolerance to 10^{-12} reduced them by one to three orders of magnitude. This indicates that the closure error is governed by the residual of the shooting, amplified by the instability of the orbits (the saddle part of the collinear points) during the integration over a period, rather than by the accuracy of `ode113`.

Remarks. Natural continuation on x_0 with an adaptive step proved sufficient for these families, so the pseudo-arclength scheme was not needed. The families were computed down to $C \simeq 3.127$–3.128 (the stopping criterion was $C < 3.130$), which covers the range of Jacobi constants common to both families required by the assignment.

In summary, single shooting based on the symmetry of the PCR3BP, combined with natural continuation on x_0 and bisection on the natural parameter, allowed the computation of the L_1 and L_2 Lyapunov families of the Earth–Moon system and of three orbit pairs with equal Jacobi constant in $[3.1370, 3.1493]$, with $|\Delta C| \leq 1.1 \cdot 10^{-11}$ and closure errors below $5 \cdot 10^{-11}$.

A MATLAB code

```
clc, clear all, close all
format long e
set(groot,'defaultLineLineWidth',2)
set(groot,'defaultAxesFontSize',15)
s = settings;
s.matlab.appearance.figure.GraphicsTheme.TemporaryValue = "light";

%% PARAMETERS DEFINITION

% Primary bodies constants
GM_Earth = 3.98600436e5; % Gravitational parameter of the Earth [km^3/sec^2]
GM_Moon = 4.902799e3; % Gravitational parameter of the Moon [km^3/sec^2]
R = 384400; % Distance between the two primary bodies [km]

% CR3BP constants
global mu
mu = GM_Moon/(GM_Moon + GM_Earth);
alpha0 = 1e-4;

% ode113 options
options = odeset('RelTol',3e-14, 'AbsTol', 1e-14);

%% LINEARIZED MOTION SOLUTION

% Lagrange points L1 and L2 computation
Ux_fun = @(x) x - (1-mu)*(x+mu)/abs(x+mu)^3 - mu*(x+mu-1)/abs(x+mu-1)^3;
```

```
int1 = [0.5 0.95];
int2 = [1.0 1.4];
xL1 = fzero(Ux_fun,int1);
xL2 = fzero(Ux_fun,int2);

% Eigenvalues and eigenvectors computation for L1
A1 = jacobian(xL1,0);
[V1, D1] = eig(A1);
lambda1 = diag(D1);
omega1 = imag(lambda1(3));
Ec1_1 = V1(:,3);

% First guess solution for L1
state0_1 = [xL1; 0; 0; 0] + alpha0*real(Ec1_1);
x01 = state0_1(1);
v01 = state0_1(4);
T01 = 2*pi/omega1;
X01 = [x01; v01; T01];

% Eigenvalues and eigenvectors computation for L2
A2 = jacobian(xL2,0);
[V2, D2] = eig(A2);
lambda2 = diag(D2);
omega2 = imag(lambda2(3));
Ec1_2 = V2(:,3);

% First guess solution for L2
state0_2 = [xL2; 0; 0; 0] + alpha0*real(Ec1_2);
x02 = state0_2(1);
v02 = state0_2(4);
T02 = 2*pi/omega2;
X02 = [x02; v02; T02];


%% FIRST ORBIT COMPUTATION

% Integration parameters
tau = [0 0.5];
eps = 1e-12;

% Orbit around L1
it1 = 0;
while true
    it1 = it1+1;
    X01_full = [x01;0;0;v01;reshape(eye(4,4),16,1)];

    % Integration of the State and STM until tau = 0.5
    [t, X] = ode113(@(t,X) Model_and_transition(t,X,T01), tau, X01_full,options);

    % Extraction of the final state
    Xf = X(end,:);

    % Constraint check
    y_half = Xf(2);
    u_half = Xf(3);
    F = [y_half; u_half];

    if abs(F(1))<eps && (abs(F(2))<eps)
        break
    end

    % Extraction of the final STM and DF computation
    PHI_half = reshape(Xf(5:20),4,4);
    DF = nonlin_jac(PHI_half,[x01;0;0;v01]);

    % Correction of the initial state
    deltaX = - DF\F;
    X01 = X01 + deltaX;
    x01 = X01(1);
    v01 = X01(2);
    T01 = X01(3);
end

```

```
100
101
102 % Orbit around L2
103 it2 = 0;
104 while true
105     it2 = it2+1;
106     X02_full = [x02;0;0;v02;reshape(eye(4,4),16,1)];
107
108     % Integration of the State and STM until tau = 0.5
109     [t, X] = ode113(@(t,X) Model_and_transition(t,X,T02), tau, X02_full,options);
110
111     % Extraction of the final state
112     Xf = X(end,:);
113
114     % Constraint check
115     y_half = Xf(2);
116     u_half = Xf(3);
117     F = [y_half; u_half];
118
119     if abs(F(1))<eps && (abs(F(2))<eps)
120         break
121     end
122
123     % Extraction of the final STM and DF computation
124     PHI_half = reshape(Xf(5:20),4,4);
125     DF = nonlin_jac(PHI_half,[x02;0;0;v02]);
126
127     % Correction of the initial state
128     deltaX = - DF\F;
129     X02 = X02 + deltaX;
130     x02 = X02(1);
131     v02 = X02(2);
132     T02 = X02(3);
133 end
134
135
136 %% NATURAL CONTINUATION
137
138 dx0 = 1e-5; % initial step (first orbit has a very small amplitude)
139 dxmax = 2e-3; % maximum step (keeps adjacent orbits close for the bisection)
140 dxmin = 1e-9; % minimum step before giving up
141 Nmax = 3000; % maximum number of orbits per family
142 Cmin = 3.130; % stop when C goes below this value
143
144 % Each row of fam is [x0 v0 T C]
145 fam1 = natural_continuation(x01, v01, T01, xL1, dx0, dxmax, dxmin, Nmax, Cmin, options);
146 fam2 = natural_continuation(x02, v02, T02, xL2, dx0, dxmax, dxmin, Nmax, Cmin, options);
147
148 fprintf('L1:␣%d␣orbits,␣C␣in␣[%.5f,␣%.5f]\n', size(fam1,1), min(fam1(:,4)), max(fam1(:,4)));
149 fprintf('L2:␣%d␣orbits,␣C␣in␣[%.5f,␣%.5f]\n', size(fam2,1), min(fam2(:,4)), max(fam2(:,4)));
150
151 %% PAIRS WITH THE SAME JACOBI CONSTANT (bisection on x0)
152
153 Cstar = [3.1380 3.1420 3.1470]; % target values inside [3.1370, 3.1493]
154 nP = numel(Cstar);
155 orb1 = zeros(nP, 4); % [x0 v0 T C] for L1
156 orb2 = zeros(nP, 4); % [x0 v0 T C] for L2
157
158 for i = 1:nP
159     orb1(i,:) = bisection_C(fam1, Cstar(i), options);
160     orb2(i,:) = bisection_C(fam2, Cstar(i), options);
161 end
162
163 % Summary table
164 fprintf('\n␣␣C*␣␣␣␣␣␣␣␣␣␣␣␣|␣L1:␣x0,␣v0,␣T,␣C␣|␣L2:␣x0,␣v0,␣T,␣C\n');
165 for i = 1:nP
166     fprintf('%.4f␣|␣%.10f␣%.10f␣%.8f␣%.12f␣|␣%.10f␣%.10f␣%.8f␣%.12f\n', ...
167         Cstar(i), orb1(i,:), orb2(i,:));
168 end
169
170 %% VERIFICATION AND PLOTS
171
172 colors = lines(nP);
```

```
173 |
174 | % Closure and Jacobi checks for the paired orbits
175 | for i = 1:nP
176 |     for j = 1:2
177 |         if j == 1, o = orb1(i,:); else, o = orb2(i,:); end
178 |         [closure, dC] = check_orbit(o(1:3), options);
179 |         fprintf('C*=%.4f␣L%d:␣closure␣=␣%.2e,␣max|C-C0|␣=␣%.2e\n', ...
180 |             Cstar(i), j, closure, dC);
181 |     end
182 | end
183 |
184 | % Window around the Moon, covering L1, L2 and the paired orbits
185 | xa = xL1 - 0.04;
186 | xb = xL2 + 0.04;
187 | ya = 0.15;
188 |
189 | % Grid for the zero-velocity curves (ZVC): 2U(x,y) = C
190 | [Xg, Yg] = meshgrid(linspace(xa, xb, 900), linspace(-ya, ya, 900));
191 | Ug = U_grid(Xg, Yg);
192 |
193 | figure(10), clf, hold on, axis equal, grid on, box on
194 | xlabel('x␣[-]'), ylabel('y␣[-]')
195 |
196 |
197 | hOrb = gobjects(nP,1);
198 | for i = 1:nP
199 |     % ZVC (dashed)
200 |     [~, hc] = contour(Xg, Yg, 2*Ug, [Cstar(i) Cstar(i)]);
201 |     hc.LineColor = colors(i,:);
202 |     hc.LineStyle = '--';
203 |     hc.LineWidth = 1.2;
204 |
205 |     % Paired orbits: same color = same C (solid)
206 |     hOrb(i) = plot_orbit(orb1(i,1:3), options, colors(i,:), 2); % L1
207 |     plot_orbit(orb2(i,1:3), options, colors(i,:), 2); % L2
208 | end
209 |
210 | % Moon and libration points
211 | plot(1-mu, 0, 'ko', 'MarkerFaceColor', [.5 .5 .5], 'MarkerSize', 9)
212 | plot(xL1, 0, 'kx', 'MarkerSize', 10, 'LineWidth', 2)
213 | plot(xL2, 0, 'kx', 'MarkerSize', 10, 'LineWidth', 2)
214 | text(xL1, -0.015, 'L_1', 'HorizontalAlignment', 'center', 'VerticalAlignment', 'top')
215 | text(xL2, -0.015, 'L_2', 'HorizontalAlignment', 'center', 'VerticalAlignment', 'top')
216 | text(1-mu, -0.015, 'Moon', 'HorizontalAlignment', 'center', 'VerticalAlignment', 'top')
217 |
218 | % Legend with the C values (proxy handles for the line styles)
219 | hO = plot(NaN, NaN, 'k-', 'LineWidth', 2);
220 | hZ = plot(NaN, NaN, 'k--', 'LineWidth', 1.2);
221 | legend([hOrb; hO; hZ], [compose('C␣=␣%.4f', Cstar(:)); {'Lyapunov␣orbits'}; {'ZVC'}], ...
222 |        'Location', 'northeastoutside')
223 |
224 | xlim([xa xb]), ylim([-ya ya])
225 | drawnow
226 |
227 |
228 | %% OUTPUT FOR THE REPORT
229 | TU = sqrt(R^3/(GM_Earth+GM_Moon))/86400; % time unit in days
230 | fprintf('\nmu␣=␣%.15e\n', mu);
231 | fprintf('xL1␣=␣%.12f␣␣␣xL2␣=␣%.12f\n', xL1, xL2);
232 | fprintf('C_L1␣=␣%.8f␣␣␣C_L2␣=␣%.8f\n', 2*potential(xL1,0), 2*potential(xL2,0));
233 | disp('lambda␣L1:'), disp(lambda1.')
234 | disp('lambda␣L2:'), disp(lambda2.')
235 | fprintf('omega1␣=␣%.10f␣␣T_lin1␣=␣%.10f\n', omega1, 2*pi/omega1);
236 | fprintf('omega2␣=␣%.10f␣␣T_lin2␣=␣%.10f\n', omega2, 2*pi/omega2);
237 |
238 | % Initial Converged Orbits
239 | fprintf('\nL1␣initial:␣x0=%.12f␣v0=%.12e␣T=%.12f␣C=%.10f␣(it=%d)\n', ...
240 |     X01(1), X01(2), X01(3), jacobi_const(X01(1),X01(2)), it1);
241 | fprintf('L2␣initial:␣x0=%.12f␣v0=%.12e␣T=%.12f␣C=%.10f␣(it=%d)\n', ...
242 |     X02(1), X02(2), X02(3), jacobi_const(X02(1),X02(2)), it2);
243 |
244 | % Families
245 | for j = 1:2
```

```
    if j==1, f = fam1; else, f = fam2; end
    fprintf('L%d: N=%d, x0 in [%.6f, %.6f], C in [%.6f, %.6f], T in [%.4f, %.4f] (%.2f-%.2f days)\n', ...
        j, size(f,1), min(f(:,1)), max(f(:,1)), min(f(:,4)), max(f(:,4)), ...
        min(f(:,3)), max(f(:,3)), min(f(:,3))*TU, max(f(:,3))*TU);
end

% Pairs with bisection C
fprintf('\nPairs:\n');
for i = 1:nP
    [~, n1] = bisection_C(fam1, Cstar(i), options);
    [~, n2] = bisection_C(fam2, Cstar(i), options);
    fprintf('C*=%.4f | dC=%.2e | bisection it L1=%d L2=%d | T1=%.4f d, T2=%.4f d\n', ...
        Cstar(i), abs(orb1(i,4)-orb2(i,4)), n1, n2, orb1(i,3)*TU, orb2(i,3)*TU);
end

%% REPORT PLOTS
cm = parula(25);

figure(11), clf, hold on, axis equal, grid on, box on
i1 = round(linspace(1,size(fam1,1),25));
i2 = round(linspace(1,size(fam2,1),25));
for j = 1:25
    plot_orbit(fam1(i1(j),1:3), options, cm(j,:), 1);
    plot_orbit(fam2(i2(j),1:3), options, cm(j,:), 1);
end
plot(1-mu,0,'ko','MarkerFaceColor',[.5 .5 .5],'MarkerSize',9)
plot([xL1 xL2],[0 0],'kx','MarkerSize',10)
xlabel('x [-]'), ylabel('y [-]')
exportgraphics(gcf,'figFamilies.png','Resolution',300)

figure(12), clf, hold on, grid on, box on
plot(fam1(:,1), fam1(:,4), 'DisplayName','L_1')
plot(fam2(:,1), fam2(:,4), 'DisplayName','L_2')
xlabel('x_0 [-]'), ylabel('C [-]'), legend
exportgraphics(gcf,'figCvsX0.png','Resolution',300)

figure(13), clf, hold on, grid on, box on
plot(fam1(:,3)*TU, fam1(:,4), 'DisplayName','L_1')
plot(fam2(:,3)*TU, fam2(:,4), 'DisplayName','L_2')
xlabel('T [days]'), ylabel('C [-]'), legend
exportgraphics(gcf,'figCvsT.png','Resolution',300)

exportgraphics(figure(10),'figPairs.png','Resolution',300)



%% LOCAL FUNCTIONS

% Jacobi integral function and derivatives
function [U, Ux, Uy, Uxx, Uyy, Uxy] = potential(x, y)
    global mu

    % Distances from the two primary bodies
    r1 = sqrt((x+mu)^2 + y^2);
    r2 = sqrt((x+mu-1)^2+y^2);

    % Potential
    U = (x^2+y^2)/2 + (1-mu)/r1 + mu/r2;

    % First derivatives
    Ux = x - (1-mu)*(x+mu)/r1^3 - mu*(x+mu-1)/r2^3;
    Uy = y - (1-mu)*y/r1^3 - mu*y/r2^3;

    % Second derivatives
    Uxx = 1 - (1 - mu)/r1^3 - mu/r2^3 ...
      + 3*(1 - mu)*(x + mu)^2/r1^5 + 3*mu*(x - 1 + mu)^2/r2^5;
    Uyy = 1 - (1 - mu)/r1^3 - mu/r2^3 ...
      + 3*(1 - mu)*y^2/r1^5 + 3*mu*y^2/r2^5;
    Uxy = 3*(1 - mu)*(x + mu)*y/r1^5 + 3*mu*(x - 1 + mu)*y/r2^5;

end

% Jacobian matrix A
```

```
function A = jacobian(x,y)
    global mu

    % Distances from the two primary bodies
    r1 = sqrt((x+mu)^2 + y^2);
    r2 = sqrt((x+mu-1)^2+y^2);

    % Partial derivatives of the potential
    Uxx = 1 - (1 - mu)/r1^3 - mu/r2^3 ...
      + 3*(1 - mu)*(x + mu)^2/r1^5 + 3*mu*(x - 1 + mu)^2/r2^5;
    Uyy = 1 - (1 - mu)/r1^3 - mu/r2^3 ...
      + 3*(1 - mu)*y^2/r1^5 + 3*mu*y^2/r2^5;
    Uxy = 3*(1 - mu)*(x + mu)*y/r1^5 + 3*mu*(x - 1 + mu)*y/r2^5;

    % Initialize the A matrix
    A = zeros(4,4);

    % First row
    A(1,3) = 1;

    % Second row
    A(2,4) = 1;

    % Third row
    A(3,1) = Uxx;
    A(3,2) = Uxy;
    A(3,4) = 2;

    % Fourth row
    A(4,1) = Uxy;
    A(4,2) = Uyy;
    A(4,3) = -2;

end

% State and transition matrix
function dX = Model_and_transition(~,X,T)
    global mu

    % Initialization (4 state elements + 16 STM elements)
    dX = zeros(20,1);

    % Extract state for readability
    x = X(1);
    y = X(2);
    u = X(3);
    v = X(4);

    % Distances from the two primary bodies
    r1 = sqrt((x+mu)^2 + y^2);
    r2 = sqrt((x+mu-1)^2+y^2);

    % First derivatives of the potential U
    Ux = x - (1-mu)*(x+mu)/r1^3 - mu*(x+mu-1)/r2^3;
    Uy = y - (1-mu)*y/r1^3 - mu*y/r2^3;

    % State integration
    dX(1) = T*u;
    dX(2) = T*v;
    dX(3) = (2*v + Ux)*T;
    dX(4) = (-2*u + Uy)*T;

    % STM integration
    % STM is 4x4 -> 16 elements -> indices from 5 to 20
    phi = X(5:20);
    PHI = reshape(phi, 4,4);

    A = jacobian(x,y);
    dphi =T*A*PHI;


    dX(5:20) = reshape(dphi, 16, 1);

```

```
end

% Jacobian matrix of the nonlinear problem
function DF = nonlin_jac(PHI, X)
    global mu

    % Initialization
    DF = zeros(2,3);

    % Extract state for readability
    x = X(1);
    y = X(2);
    u = X(3);
    v = X(4);

    % Distances from the two primary bodies
    r1 = sqrt((x+mu)^2 + y^2);
    r2 = sqrt((x+mu-1)^2+y^2);

    % First derivatives of the potential U
    Ux = x - (1-mu)*(x+mu)/r1^3 - mu*(x+mu-1)/r2^3;
    Uy = y - (1-mu)*y/r1^3 - mu*y/r2^3;

    % State at X0
    f0 = [ u;v; (2*v + Ux); (-2*u + Uy)];


    % First row
    DF(1,1) = PHI(2,1);
    DF(1,2) = PHI(2,4);
    DF(1,3) = 0.5*PHI(2,:)*f0;

    % Second row
    DF(2,1) = PHI(3,1);
    DF(2,2) = PHI(3,4);
    DF(2,3) = 0.5*PHI(3,:)*f0;

end


function C = jacobi_const(x0,v0)
    U = potential(x0,0);
    C = 2*U - v0^2;
end


% Single shooting with FIXED x0: unknowns are [v0; T]
function [v0, T, ok] = shooting_fixed_x0(x0, v0, T, options)
tolF = 1e-12; % tolerance on the constraint vector
maxIter = 20; % maximum number of Newton iterations
ok = false;
tau = [0 0.5];

for it = 1:maxIter
    % Reject non-physical periods (T <= 0 is also the trivial solution)
    if T <= 0 || ~isfinite(T) || ~isfinite(v0)
        return
    end

    X0_full = [x0; 0; 0; v0; reshape(eye(4,4),16,1)];

    % Integration of the state and STM until tau = 0.5
    [~, X] = ode113(@(t,X) Model_and_transition(t,X,T), tau, X0_full, options);
    Xf = X(end,:);

    % Constraint vector: y(T/2) = 0, u(T/2) = 0
    F = [Xf(2); Xf(3)];

    if norm(F, inf) < tolF
        ok = true;
        return
    end

```

```
    % STM at tau = 0.5 and Jacobian DF (2x3)
    PHI_half = reshape(Xf(5:20),4,4);
    DF = nonlin_jac(PHI_half, [x0; 0; 0; v0]);

    % x0 is fixed: drop its column, solve the 2x2 system in [v0; T]
    d = -DF(:,2:3) \ F;
    v0 = v0 + d(1);
    T = T + d(2);
end
end

% Natural continuation on x0 with adaptive step
function fam = natural_continuation(x0, v0, T, xL, dx0, dxmax, dxmin, Nmax, Cmin, options)
% Step direction: away from the libration point
s = sign(x0 - xL);

fam = zeros(Nmax, 4);
fam(1,:) = [x0 v0 T jacobi_const(x0, v0)];
k = 1;
dx = dx0;

while k < Nmax && fam(k,4) > Cmin
    xn = fam(k,1) + s*dx;

    % Initial guess for [v0 T]
    if k >= 2
        % Secant predictor, scaled by the ratio of the steps
        hprev = abs(fam(k,1) - fam(k-1,1));
        g = fam(k,2:3) + (fam(k,2:3) - fam(k-1,2:3)) * (dx/hprev);
    else
        % First step: v0 is proportional to (x0 - xL) in the linear regime
        g = [fam(k,2)*(xn - xL)/(fam(k,1) - xL), fam(k,3)];
    end

    [vn, Tn, ok] = shooting_fixed_x0(xn, g(1), g(2), options);

    % Sanity check: the period must stay close to the previous one
    % (avoids jumping to another family or to the trivial solution)
    if ok && (Tn <= 0 || abs(Tn - fam(k,3)) > 0.1*fam(k,3))
        ok = false;
    end

    if ~ok
        % Halve the step and retry from the same orbit
        dx = dx/2;
        if dx < dxmin
            warning('NC: step below dxmin at k = %d (x0 = %.6f)', k, fam(k,1));
            break
        end
        continue
    end

    % Accept the orbit and enlarge the step (up to dxmax)
    k = k + 1;
    fam(k,:) = [xn vn Tn jacobi_const(xn, vn)];
    dx = min(1.5*dx, dxmax);
end

fam = fam(1:k,:);
end

% Bisection on x0 to find the orbit of the family with C = Cstar
function [sol, nit] = bisection_C(fam, Cstar, options)
C = fam(:,4);

% Find two consecutive orbits bracketing Cstar
k = find((C(1:end-1) - Cstar) .* (C(2:end) - Cstar) <= 0, 1);
assert(~isempty(k), 'Cstar = %.4f is outside the family range', Cstar);

a = fam(k,:); % [x0 v0 T C]
b = fam(k+1,:);
tolC = 1e-11;

```

```
for it = 1:100
    nit = it;

    % Midpoint on the natural parameter
    xm = 0.5*(a(1) + b(1));

    % Guess for [v0 T]: average of the two bracketing orbits
    g = 0.5*(a(2:3) + b(2:3));

    % Recompute the orbit at xm with the shooting
    [vm, Tm, ok] = shooting_fixed_x0(xm, g(1), g(2), options);
    assert(ok, 'Shooting did not converge during bisection');

    Cm = jacobi_const(xm, vm);
    sol = [xm vm Tm Cm];

    % Stop on tolerance (or if the interval is at machine precision)
    if abs(Cm - Cstar) < tolC || abs(b(1) - a(1)) < 1e-15
        return
    end

    % Keep the sub-interval that still brackets Cstar
    if (Cm - Cstar) * (a(4) - Cstar) > 0
        a = sol;
    else
        b = sol;
    end
end
end

% Integrate one full period and plot the orbit in the (x,y) plane
function h = plot_orbit(o, options, col, lw)
    X0 = [o(1); 0; 0; o(2); reshape(eye(4),16,1)];
    [~, X] = ode113(@(t,X) Model_and_transition(t,X,o(3)), linspace(0,1,400), X0, options);
    h = plot(X(:,1), X(:,2), 'Color', col, 'LineWidth', lw);
end

% Closure error and Jacobi constant drift over one full period
function [closure, dC] = check_orbit(o, options)
    X0 = [o(1); 0; 0; o(2); reshape(eye(4),16,1)];
    [~, X] = ode113(@(t,X) Model_and_transition(t,X,o(3)), linspace(0,1,400), X0, options);

    % Closure: state at tau = 1 must match the initial state
    closure = norm(X(end,1:4) - X(1,1:4));

    % Jacobi constant along the orbit: C = 2U - (u^2 + v^2)
    Cvec = zeros(size(X,1),1);
    for i = 1:size(X,1)
        Cvec(i) = 2*potential(X(i,1), X(i,2)) - (X(i,3)^2 + X(i,4)^2);
    end
    dC = max(abs(Cvec - Cvec(1)));
end

% Vectorized potential U(x,y) for matrix inputs (used for the ZVC)
function U = U_grid(x, y)
    global mu
    r1 = sqrt((x+mu).^2 + y.^2);
    r2 = sqrt((x+mu-1).^2 + y.^2);
    U = (x.^2 + y.^2)/2 + (1-mu)./r1 + mu./r2;
    U(~isfinite(U)) = NaN;
end
```